

Midterm Project: Diagnose and Repair a Company's Network Services

Server Operating System (192-442)

Overview

Your group has taken over the IT operations of a small company whose core network services -- DNS and DHCP -- are **misconfigured and broken**. A machine that joins this company's network with no manual configuration currently **cannot resolve the company's own host names**. Your job is to find every fault, prove each one with evidence, fix them, and document how you did it.

This is a group project (3-5 students). Total: 100 marks.

The marks are for **diagnosis and evidence**, not for typing configuration.

Anyone -- or any AI -- can produce a correct config from a specification. The skill this project measures is the one that matters on a real server: reasoning from an observed symptom to its cause, and proving you were right.

Your Environment

You work in a KillerCoda playground where you are **root**. When it starts, it automatically builds the company's (broken) DNS + DHCP setup and a **test client** on the network. The lab uses a synthetic network segment on the interface **lan0** (not eth0). A client machine sits in a separate namespace and must obtain everything -- address, gateway, resolver, domain -- by DHCP.

You are root, so you can read the checking tools in `/root/grading`. Reading them tells you **what** is checked; it does not find or fix the faults for you, and the marks are for your evidence.

The Goal, Stated Precisely

A machine joining this network with no manual configuration must be able to resolve the company's own host names.

Right now it cannot. There are **several faults** standing in the way. Some are in configuration files. Some are not -- a fault can live in file ownership, in `/etc/default/...`, or in a firewall rule that no configuration file would ever show. Do not assume `git`-style thinking; look at the running system.

Part 1: Diagnose and Repair (50 marks)

Use the checker to see where you stand. It tests from the **client's** point of view and tells you what it observed versus what it expected:

```
python3 /root/grading/check_lab.py --group lab01 --phase build
```

Work from each symptom to its cause. A few honest pointers:

- `dig @127.0.0.1` on the server proves almost nothing. The server can look perfect to itself and still be unreachable from the client. Test the way the checker does -- from the client.

- Distinguish the failure modes. A **timeout** is not the same as **REFUSED**, which is not the same as **SERVFAIL**, which is not the same as **NXDOMAIN**, which is not the same as **NOERROR with no answer**. Each points at a different kind of cause.

- "Works on the server, fails from the client" is the signature of a reachability fault (listen address, allow-query, or a firewall rule).

Fix each fault in the real system. After each change, restart the affected service and re-run the checker. Marks are awarded by the checker for the outcomes it can verify (the service works, the client leases, the client resolves the company's names using only what the lease advertised).

Part 2: The Incident Log (20 marks)

As you work, keep an **incident log** in `INCIDENT_LOG.md` -- a record of HOW you found and proved each fault, not just what the fix was. For each fault:

- **Symptom:** what you observed (paste the actual command output).
- **Hypothesis:** what you suspected and why.
- **Proof:** the command whose output confirmed the cause (paste it).
- **Fix:** what you changed.
- **Confirmation:** the output showing it now works.

An entry that just says "changed X to Y" earns little. An entry that shows the diagnostic path -- the wrong output, the command that revealed the cause, the corrected output -- earns full marks. Work inside a recorded session so your process is captured:

```
script -a /root/session.log
```

Commit both `INCIDENT_LOG.md` and `session.log`.

Part 3: AI Usage and Critique (30 marks)

Modern administrators use AI. This project does not forbid it -- it asks you to use it **honestly and critically**, and to prove you understand where it helps and where it fails. Record everything in `AI_CRITIQUE.md`.

3a. Declaration (required)

List every AI tool your group used anywhere in this project, and how. Declaring

is required; using AI is allowed. Undeclared use found in grading is an academic-honesty issue.

3b. Do your own diagnosis FIRST

Complete your incident log (Part 2) from your own work before doing the AI comparison below. The comparison is only meaningful once you have found the faults yourselves.

3c. Put the AIs to the test

Give the broken scenario (or a description of a symptom) to **at least two** different AI tools (for example Claude, Gemini, ChatGPT, Copilot). Ask each to diagnose and fix. Record what each produced, then evaluate:

Fault (in your words)	You found it?	AI #1	AI #2	Was each AI actually right?
-----------------------	---------------	-------	-------	-----------------------------

The last column matters most. AI confidently produces fixes that are wrong, irrelevant, or that "fix" a symptom without addressing the cause. For each AI suggestion, state whether it was correct and how you verified that.

3d. Verdict

In a few short paragraphs: where did your human diagnosis beat the AIs? Where did an AI genuinely help? What did you learn about trusting AI to diagnose a live system it cannot actually see?

Marks are for the QUALITY of your critical evaluation -- especially 3c (was the AI actually right?) and 3d -- not for how many AIs you tried.

Submitting

When your checker score is as high as you can get it, submit to your group's GitHub repository:

- your fixed configuration (or a copy of the relevant files)
- `INCIDENT_LOG.md`
- `session.log` (your recorded session)
- `AI_CRITIQUE.md`

Each group member must have commits under their own git identity, showing the part of the work they did.

Marking Rubric

Part	Component	Marks
1	Working DNS + DHCP, verified by the live client checker	50
2	Incident log: how each fault was found and proved	20
3	AI usage declaration, comparison, and critique	30

	Total	100
--	--------------	------------

Part 1 is scored automatically against outcomes on a real client. Parts 2 and 3 are read and graded by your instructor. You must have a genuinely working system and your own diagnosis to do Parts 2 and 3 meaningfully.

Individual contribution is assessed from commit history and authorship. A member with no commits may receive a reduced mark.